How to KISS
and be happy about it

Rosen Vitanov

07.10.2026, Aachen

What is KISS?

Keep it simple, stupid
(super simple)

For this talk:

behavior that is easy to predict

and code that is easy to change.

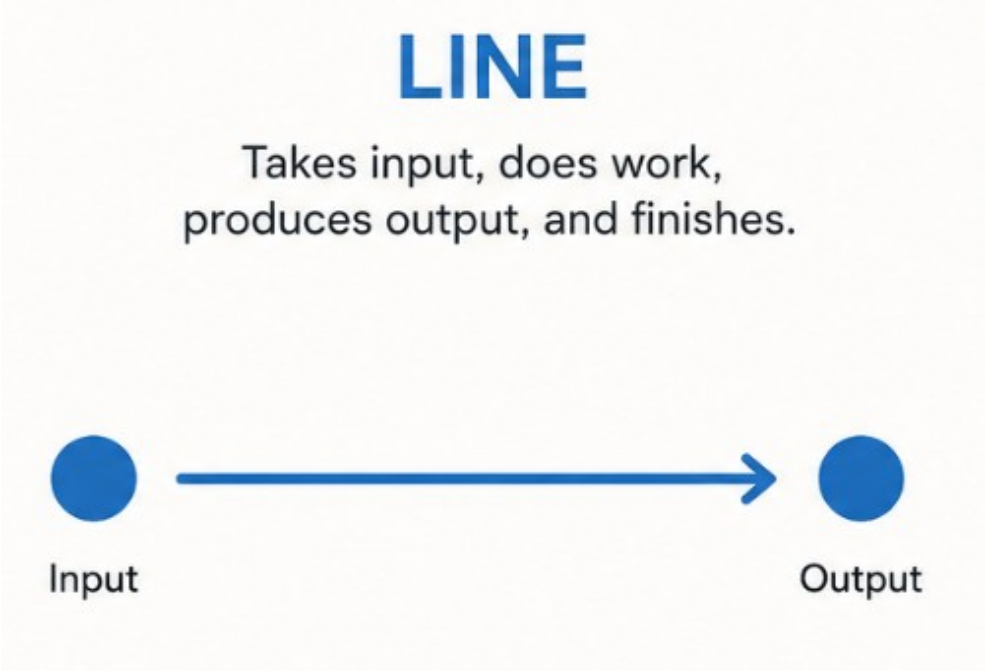
The cost of remembering

“Good Programming is not about how smart you are.

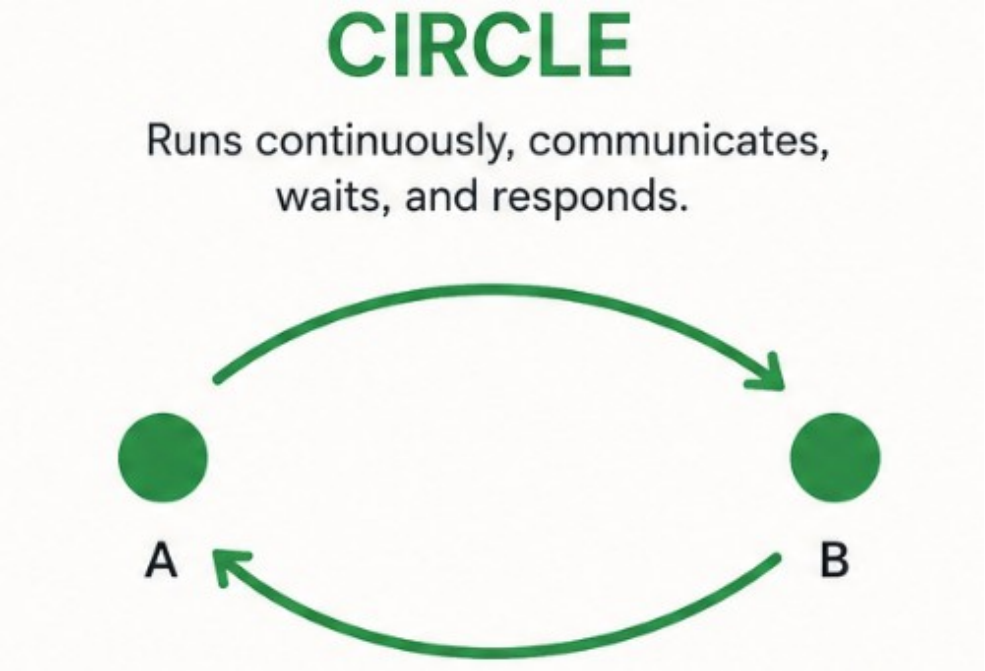
It’s about how much future you has to remember.”

Two useful mental models

A transformation



A conversation



Feedback contains transformations

A line transforms input into output.

A circle responds to events and feeds results back.

The cycle contains smaller, individual transformations.

Requirements often tell you which model fits the task.

Incomplete requirements

Requirements give us a starting point.

They can omit edge cases and hide assumptions.

They often describe only part of the process or the happy path.

They change as we learn.

Guessing intent

“We need to check if a file exists.”

“I’ll make a distributed filesystem abstraction layer.”

Clarify the need before choosing the abstraction.

Another example: “faster signup” does not necessarily mean removing verification.

Don’t repeat yourself

“Show every ID as internal_id + 1.”

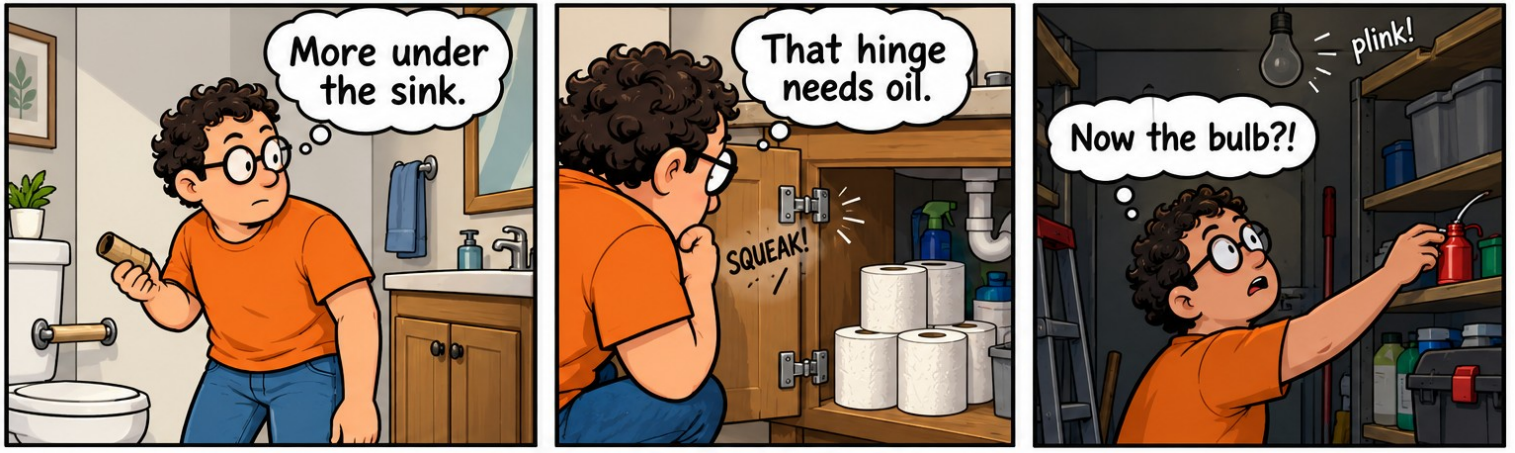
One rule becomes several implementations.

A change then has to reach every copy.

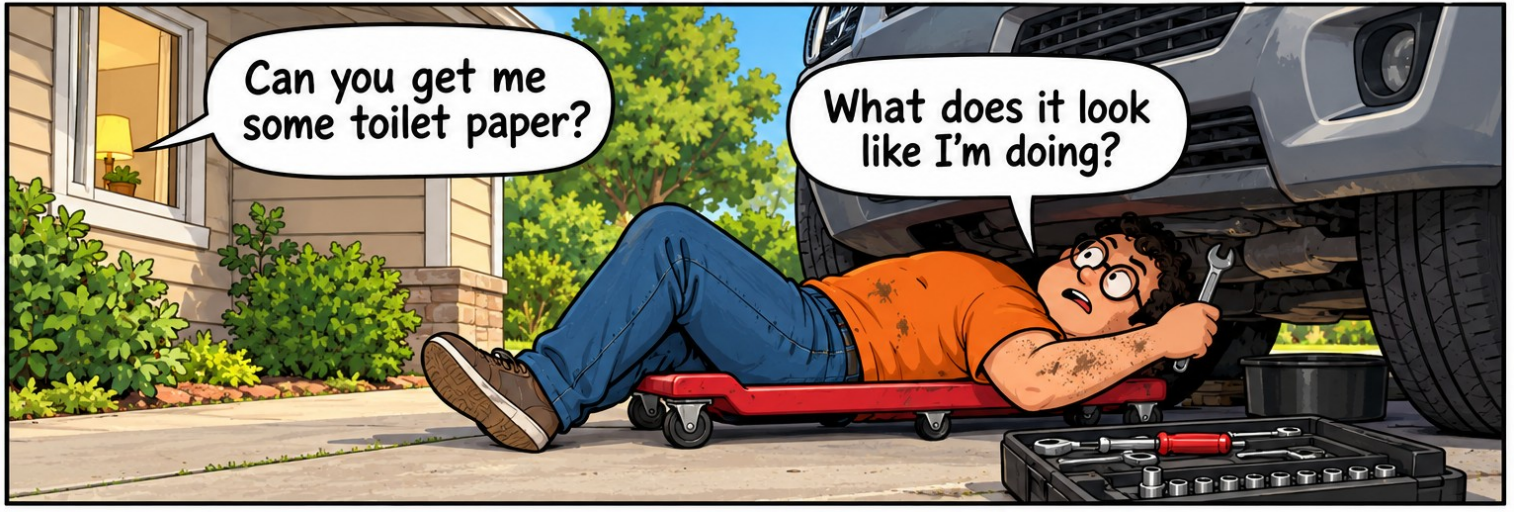
Prefer: One rule, one authoritative definition



Don’t forget the toilet paper



Several side tasks later...



Focus on the pressing task

Try to finish the main task first

Having many ideas is great.

Make notes as you go, but stay on the current path.



Obsolete behavior

“We removed the feature, except from the codebase.”

Old paths preserve assumptions that no longer apply.

Predictable behavior

Guessing intent, duplicated rules, obsolete behavior

Simple code is easier to predict and easier to redirect when requirements change.

Stale or incomplete assumptions make both harder.

Refactoring pain signals

“I don’t know where the bug is.”

“Changing one thing breaks five.”

“I need to understand the whole system first.”

“I’m afraid to touch it.”

Also: too many special cases, painful tests, and recurring bugs.

Where do we start?

Free functions expose transformations

A small transformation can be hidden inside a large class or inheritance hierarchy.

Explicit inputs and outputs make dependencies visible.

Extraction can reduce duplication and clarify the code.

These functions are mostly easily testable.

Smaller data structures

Large classes can mix unrelated responsibilities.

Many members and inheritance levels make changes harder to reason about.

Coupling also makes tests harder to set up.

```
struct RuntimeData {
  string addr;
  float  x;
  int    cap;          // size
  int    tries;
  int    port;
  string key;
  float  h;
  int    delay;        // time
  bool   valid;
  string proto;
  int    max;
  float  scale;
  int    used;
  bool   again;
  string cert;
  float  y;
};
```

And messy gets messier over time

```
struct RuntimeData {
  string addr;
  float  x;
  int    cap;          // size
  int    tries;
  int    port;
  string key;
  float  h;
  int    delay;        // time
  bool   valid;
  string proto;
  int    max;
  float  scale;
  int    used;
  bool   again;
  string cert;
  float  y;
  int  ttl;            // time
  int  n;
  float w;
  string proxy;
  bool  dirty;
  int   wait;
  float angle;
  string token;
  int   hits;
  bool  stop;
  float dx;
  bool  secure;
  int   misses;
  float dy;
  int   factor;        // multiplier
  bool  enabled;       // enabled
};
```

Separate data by responsibility

```
struct ConnectionConfig {
   string  addr;
   int     port;
   string  proto;
   string  cert;
   string  proxy;
   string  token;
   bool    secure;
   bool    enabled;
};
```

```
struct Geometry {
   float  x;
   float  y;
   float  w;
   float  h;
   float  scale;
   float  angle;
   float  dx;
   float  dy;
};
```

```
struct CacheState {
   int     cap;
   string  key;
   bool    valid;
   int     used;
   int     ttl;
   bool    dirty;
   int     hits;
   int     misses;
};
```

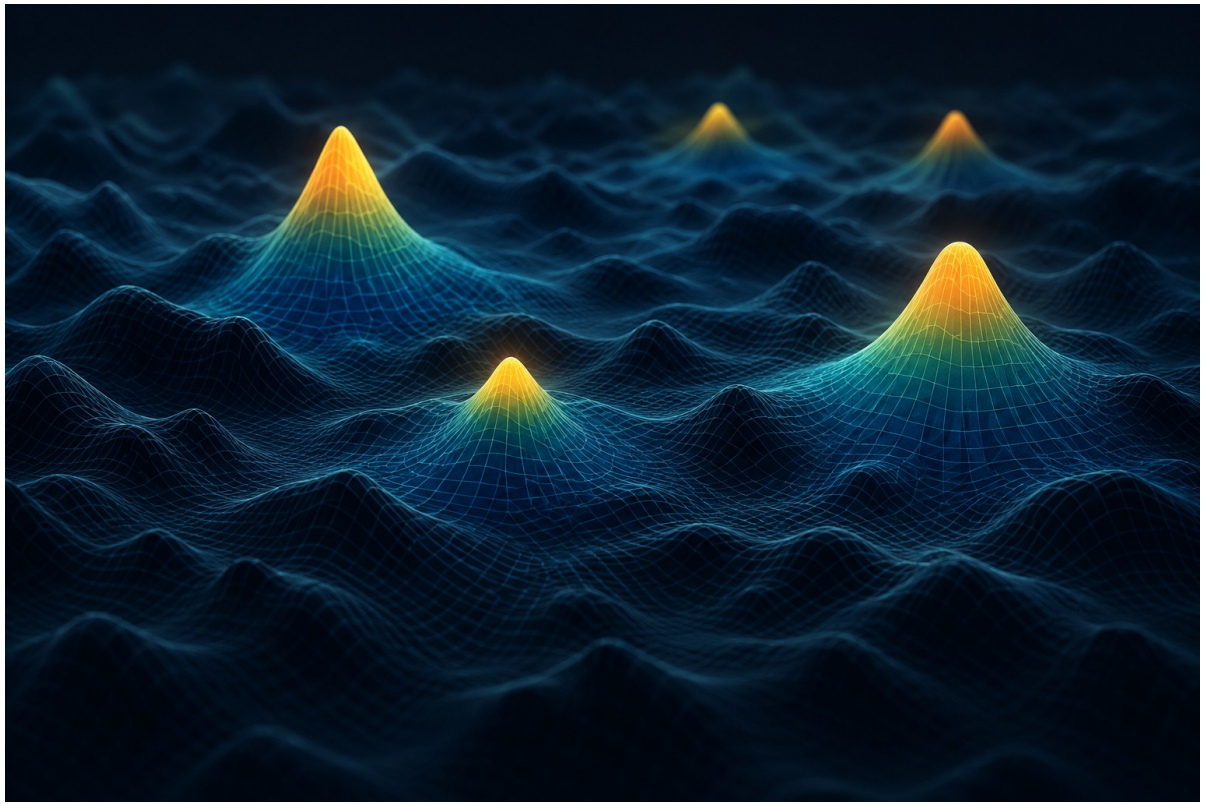
```
struct RetryPolicy {
   int   tries;
   int   delay;
   int   max;
   bool  again;
   int   n;
   int   wait;
   bool  stop;
   int   factor;
};
```

Inspiration from other sciences

“It is important to draw wisdom from many different places. If you take it from one place, it becomes rigid and stale.”

Two analogies for thinking about software design

An implementation landscape



A feature can have many possible implementations.

Moving between designs can require difficult intermediate changes.

Small transformations with explicit dependencies can be easier to carry across.

Similar problems, recurring patterns



Nature offers examples of similar solutions to similar pressures.

In software, design patterns describe recurring solutions.

Different implementations still involve different trade-offs.

When construction gets complicated

A transformation may need substantial configuration or several steps to prepare its inputs.

A builder can collect and cache configuration before creating the result.

Free functions can provide convenient, named configurations.

This is one way to combine OOP and functional programming.

The examples illustrate the pattern, rather than a universal rule.

Builder example: a named configuration

```
Burger classic_burger() {
    return BurgerBuilder{}
        .addBun(Bun::Sesame)
        .addPatty(Patty::Beef)
        .addCheese(Cheese::Cheddar)
        .addTopping(Topping::Lettuce)
        .addTopping(Topping::Tomato)
        .build();
}
```

The builder collects the choices, caches performance critical data and produces the final result.

The free function names a useful recipe.

This is a bad example, since this boils down to an aggregate initialization.

You don’t really need a Builder for this.

Builder example: a scheduled task

```
TaskScheduler make_repeated_schedule(
    std::chrono::seconds interval,
    std::function<void()> task) {
    return TaskSchedulerBuilder{}
        .interval(interval, std::move(task))
        .makeRepeatedSchedule()
        .setupBackgroundTasks()
        .build();
}
```

The helper exposes the choices callers need.

The builder caches required data during construction and produces one final result.

Choosing a representation

Need	Useful starting point
Simple transformation	Free function(s)
Complex construction + cache	Builder + free functions
Data	Small data objects / structs

These are design preferences to evaluate in context.

Data and logic have different roles

Data objects describe the information.

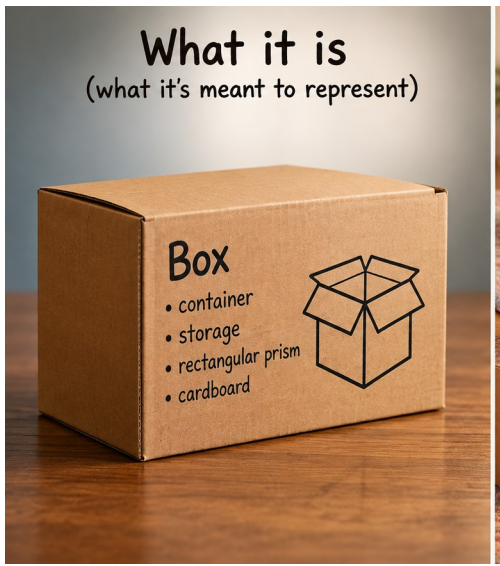
Free functions express operations on that information.

Small, reusable structs can reduce unnecessary coupling.

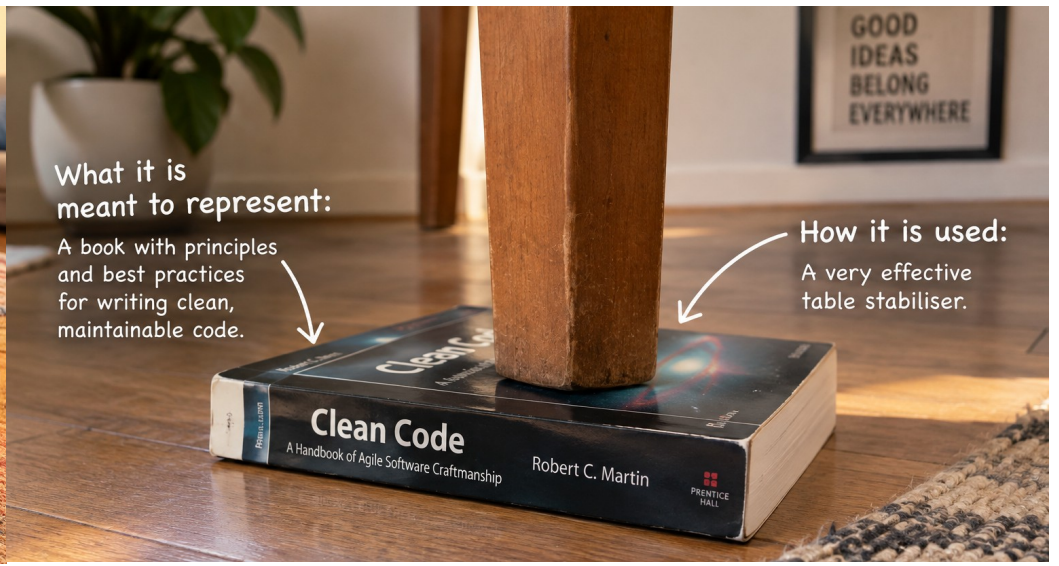
Separate data and logic as much as possible.

What it is and how it is used

The same object can serve purposes its designer never imagined.







The smallest useful dependency

A function that needs only a width can accept only a width.

```
TableLeg leg;
Book cleanCode;

lift_by(leg, cleanCode.height); // Ok
lift_by(leg, cleanCode);        // Unneeded coupling
```

A whole Book object would couple this operation to information it does not use.

Simple data works well with projections

```
struct Person {
    std::string firstName;
    std::string lastName;
    int age;
};

std::ranges::sort(people, {}, &Person::age);
std::ranges::find(people, “Bjarne”, &Person::firstName);
```

The projection identifies the field used for sorting.

The data stays easy to construct in tests.

Data ownership and invariants

“When A changes, B must also change.”

Make responsibility for that rule explicit.

A data owner can coordinate updates.

Use encapsulation when it helps protect an invariant.

Some Practical Examples

Enough philosophy...

Function complexity

Signature	Possible Behaviors
`int foo()`	1*
`int foo(bool)`	2
`int foo(bool, bool)`	4
`int foo(int8_t)`	256

For independent on/off options, a flags utility gives those choices names.

A reusable Flags<Enum> utility is very helpful

```
enum class Permission : unsigned {
    Read = 0x1, Write = 0x2, Execute = 0x4
};

// Note: overload of operator| producing a flags class from the enum
Flags<Permission> access{Permission::Read | Permission::Write};
Flags<Permission> requested{Permission::Write | Permission::Execute};

access.hasFlags(requested);    // false: not all
access.hasAnyFlags(requested);  // true: Write
```

The custom template provides the reusable utility, avoiding complex flag logic in code. Implementation of Flags class left as an exercise.

Guard clauses keep the main path clear

```
Flags<Permission> get_edit_permissions(
    const User& user, const Document& document) {
    if (document.locked)
        return {};

    if (!user.access.hasFlag(Permission::Write))
        return {};

    return document.owner == user.id ?
        {Permission::Write | Permission::Comment}
      : {Permission::Comment};
}
```

Each early return answers one question.

The final line shows the remaining rule.

Useful when nesting hides the main path. Keep checks in their intended order.

Optional makes absence explicit

```
std::optional<int> first_even(
    const std::vector<int>& values) {
    for (int value : values)
        if (value % 2 == 0) return value;
    return std::nullopt;
}

if (auto value = first_even(values)) {
    consume(*value);
}
```

No special integer means “missing.”

Zero remains a valid result.

Use optional for an ordinary missing value. Errors may need a reason as well.

std::variant models mutually exclusive states

```
struct StateA {};
struct StateB { int itemCount; };
struct StateC { std::string name; };

using SystemState =
    std::variant<StateA, StateB, StateC>;

As of C++23 std::expected can also be used for
simple “value or error” cases-
```

One active state with its own data.

No combination of isLoading, isReady, and hasFailed to keep consistent.

Use a variant when alternatives carry different data.

For state labels alone, an enum is enough.

One last KISS

Keep behavior predictable.

Identify dependencies explicitly.

Split work into small transformations.

Separate data from logic.

A reward for the constexperts

```
consteval int compile_time_impl(int x) {
    return x * 2;
}


constexpr int calculate(int x) {
    return compile_time_impl(x);
}

int main() {
    constexpr int value = calculate(2);
    return value;
}
```

Will this compile?

- No. Can’t pass runtime value x to consteval.

std::is_constant_evaluated()

```
consteval int compile_time_impl(int x) {
    return x * 2;
}


constexpr int calculate(int x) {
   if (std::is_constant_evaluated()) {
       return compile_time_impl(x);
   }
   else {
       return x * 2;
   }
}
```

Will this compile?

- No. Still can’t pass runtime value x to consteval.

if consteval

```
consteval int compile_time_impl(int x) {
    return x * 2;
}


constexpr int calculate(int x) {
   if consteval {
       return compile_time_impl(x);
   }
   else {
       return x * 2;
   }
}
```

This works and compiles

std::is_constant_evaluated()

Still useful in branch conditions

```
constexpr int doMath(const Data& data, std::size_t size) {

    return std::is_constant_evaluated() || size < 16 ?
       simple_algorithm(data, size)
     : optimized_algorithm(data, size)

}
```

Some autogenerated example images

Take these with a grain of salt...

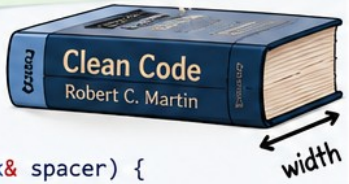
Simple, explicit and appropriate

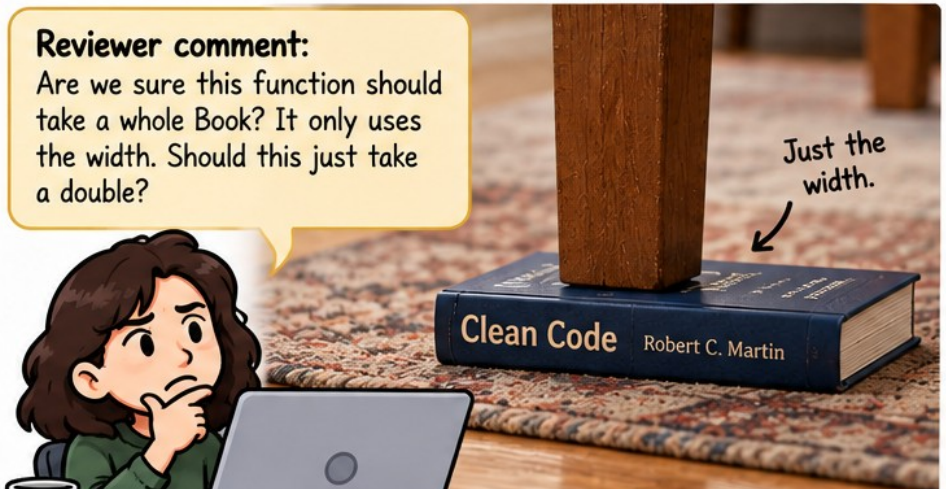
A small, focused type. We pass a Book because we already have one, and we only need its width for the spacer. The function is explicit about its intent.

```
struct Book {
    std::string title;
    std::string author;
    int pages;
    double width;    // in millimeters
    double height;   // in millimeters
    double thickness;  // in millimeters
};


void raiseTableLeg(TableLeg& leg, const Book& spacer) {
   // Use the book's width as a spacer.
   leg.raise(spacer.width);
}
```

We have a Book already (e.g. picked from a stack). Using its width here is clear and intentional.





Overengineered and obscured

The width usage is hidden inside a large, generic Book class with many unrelated responsibilities. The same simple operation is harder to see and comes with extra baggage.

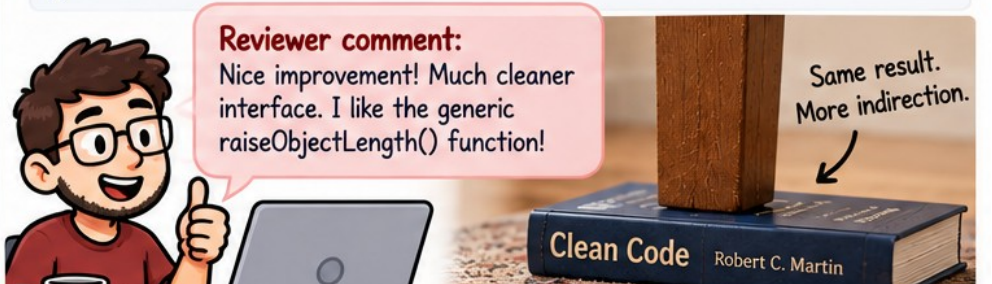
```
class Book {
public:
    // Lots of unrelated stuff...
    std::string title() const { return title_; }
    std::string author() const { return author_; }
    int pageCount() const { return pages_; }
    std::string readChapter(int n) const;
    void bookmark(int page);
    bool loadFromFile(const std::string& path);
    void syncWithCloud();
    std::vector<std::string> tags() const;
    std::optional<std::string> summary() const;
    // ... and many more ...

    // Generic utility that happens to adjust any object
    // using the book's width. Nothing book-specific here.
    template <typename T>
    void raiseObjectLength(T& obj) const {
       obj.raise(getWidth());  // uses the book's width internally
    }

private:
    double getWidth() const { return dimensions_.width; }
    struct Dimensions { double width, height, thickness; } dimensions_;
    std::string title_, author_;
    int pages_{0};
    // ... more unrelated members ...
};
```

Unrelated responsibilities:

- metadata
- file I/O
- bookmarking
- cloud sync
- tags
- summaries
- parsing
- statistics
- formatting
- search
- printing
- exporting
- ...



Builder class with fluent interface

Offers a few methods and caches the configuration internally.

class BurgerBuilder

```
public:
    BurgerBuilder& addBun(Bun bun) {
        bun_ = bun;
        return *this;
    }
    BurgerBuilder& addPatty(Patty patty) {
        patties_.push_back(patty);
        return *this;
    }
    BurgerBuilder& addCheese(Cheese cheese) {
        cheeses_.push_back(cheese);
        return *this;
    }
    BurgerBuilder& addTopping(Topping topping) {
        toppings_.push_back(topping);
        return *this;
    }

    Burger build() const {
        return Burger{bun_, patties_, cheeses_,
                toppings_};
    }

private:
    Bun bun_ = Bun::Sesame;
    std::vector<Patty> patties_;
    std::vector<Cheese> cheeses_;
    std::vector<Topping> toppings_;
};
```

Internal cached state

Bun



bun_
(default: Sesame)

Patties



patties_
(vector)

Cheeses



cheeses_
(vector)

Toppings



toppings_
(vector)

Free functions for specific burgers

Convenience helpers built using the builder.

Burger classicBurger()

```
return BurgerBuilder()
      .addBun(Bun::Sesame)
      .addPatty(Patty::Beef)
      .addCheese(Cheese::Cheddar)
      .addTopping(Topping::Lettuce)
      .addTopping(Topping::Tomato)
      .build();
```



Classic Burger

Burger baconBurger()

```
return BurgerBuilder()
      .addBun(Bun::Brioche)
      .addPatty(Patty::Beef)
      .addCheese(Cheese::Cheddar)
      .addTopping(Topping::Bacon)
      .addTopping(Topping::Onion)
      .build();
```



Bacon Burger

Burger veggieBurger()

```
return BurgerBuilder()
      .addBun(Bun::WholeGrain)
      .addPatty(Patty::Veggie)
      .addCheese(Cheese::Swiss)
      .addTopping(Topping::Lettuce)
      .addTopping(Topping::Tomato)
      .build();
```



Veggie Burger

TaskSchedulerBuilder

Configure timing, execution and error handling. Settings are cached internally until build().

```
class TaskSchedulerBuilder {
public:
    TaskSchedulerBuilder& name(std::string n) { name_ = std::move(n);
       return *this; }
    TaskSchedulerBuilder& interval(std::chrono::milliseconds i) {
       interval_ = i; return *this; }
    TaskSchedulerBuilder& initialDelay(std::chrono::milliseconds d) {
       initialDelay_ = d; return *this; }
    TaskSchedulerBuilder& repeat(bool r) {
       repeat_ = r; return *this; }
    TaskSchedulerBuilder& task(std::function<void()> t) {
       task_ = std::move(t); return *this; }
    TaskSchedulerBuilder& onError(std::function<void(std::exception_ptr)> h) {
       onError_ = std::move(h); return *this; }
    TaskSchedulerBuilder& runOnBackgroundThread(bool b) {
       background_ = b; return *this; }

    TaskScheduler build() const {
       return TaskScheduler{name_, interval_, initialDelay_, repeat_,
                            task_, onError_, background_};
    }
private:
    std::string name_ = "unnamed";
    std::chrono::milliseconds interval_ = std::chrono::seconds(1);
    std::chrono::milliseconds initialDelay_ = std::chrono::milliseconds(0);
    bool repeat_ = true;
    std::function<void()> task_ = []{};
    std::function<void(std::exception_ptr)> onError_ = nullptr;
    bool background_ = true;
};
```

Internal cached configuration

- **name_** (std::string)
- **interval_** (std::chrono::milliseconds)
- **initialDelay_** (std::chrono::milliseconds)
- **repeat_** (bool)
- **task_** (std::function<void()>)
- **onError_** (std::function<void (std::exception_ptr)>)
- **background_** (bool)

Free functions for common schedules

Convenience helpers that use the builder with different configurations.

runEvery(std::chrono::seconds, std::function<void()>)

```
TaskScheduler runEvery(std::chrono::seconds interval,
                       std::function<void()> task)
{
    return TaskSchedulerBuilder()
        .name("repeat-task")
        .interval(interval)
        .repeat(true)
        .task(std::move(task))
        .runOnBackgroundThread(true)
        .build();
}
```

Repeated task
Runs the task at a fixed interval. Useful for polling, metrics, cleanup, ...

runOnceAt(std::chrono::system_clock::time_point, std::function<void()>)

```
TaskScheduler runOnceAt(std::chrono::system_clock::time_point when,
                        std::function<void()> task)
{
    auto now = std::chrono::system_clock::now();
    auto delay = std::chrono::duration_cast<std::chrono::milliseconds>(when - now);
    if (delay < std::chrono::milliseconds(0))
        delay = std::chrono::milliseconds(0);

    return TaskSchedulerBuilder()
        .name("one-shot")
        .initialDelay(delay)
        .repeat(false)
        .task(std::move(task))
        .build();
}
```

One-shot task
Runs the task once at a specific time.

runAfter(std::chrono::seconds, std::function<void()>)

```
TaskScheduler runAfter(std::chrono::seconds delay,
                       std::function<void()> task)
{
    return TaskSchedulerBuilder()
        .name("delayed-task")
        .initialDelay(delay)
        .repeat(false)
        .task(std::move(task))
}       .build();
```

Delayed task
Runs the task once after a delay.